

1. Introduction

Our project was reimplementing the paper “A Watermark for Large Language Models” by Kirchenbauer, Geiping, Wen, Katz, Miers, and Goldstein. We chose to investigate this project as distinguishing text written by a human from text generated by AI has become increasingly difficult during the rapid improvement of LLMs, and yet is very desirable. The primary contribution of this paper is proposing a watermarking scheme, consisting of an algorithm to run on top of a model during generation to produce the watermark and a detection algorithm that can identify whether text was generated with this watermark with high accuracy.

The generation algorithm only modifies the logits of the next token, not the weights of the model itself, which means it can be run on top of a model without the need for any new training. In particular, at each step of generation, the algorithm produces a watermark through the use of a “green list.” Based on an initial randomness seed, and the context (some number of the previous tokens), the algorithm produces a pseudorandom green list consisting of a fraction (hyperparameter γ) of the vocabulary, and prioritizes generation of these tokens.

The detection algorithm looks at the number of tokens in the generated sequence which came from the green list. It can reconstruct the green list for each token since it is given the same randomness seed and combines this with the context (previous tokens). If an unusually large fraction of the generated tokens are from the green list, it identifies the text as watermarked.

2. Chosen Result

The paper discusses two versions of the watermark: one with a soft green list and one with a hard green list. The difference comes when choosing the next token after computing the green list; the hard version *always* selects a token from the green list, while the soft version boosts the probability of all green list tokens to favor them over red list tokens, but still allows red list tokens to maintain some probability of being selected. It does this by adding some δ to the logits of green tokens before we take the softmax to arrive at the probability distribution. We chose to implement the soft green list watermark from the paper, as it is more robust in real scenarios since allowing red tokens sometimes makes it better for low-entropy cases (when there are only a few reasonable choices for the next token). The disadvantage of the soft green list, as noted in the paper, is that it makes watermarked text slightly harder to identify. While the hard watermark guarantees that any text generated with at least 16 tokens will have a z-score of 4, for the soft watermark we do not have the same guarantee since it will generate some red tokens too. To test this, and evaluate how identifiable the soft watermark is for different parameter choices and sequence lengths, we aimed to reproduce the author’s figures of z-score vs number of generated tokens for different choices of δ (how much we add to the logits of green tokens) and γ (what percentage of the vocabulary is in the green list at each step of generation). That is, we reproduced graphs (a) and (b) in the below figure not reproducing (c) as we chose to only implement multinomial sampling. We decided to focus on evaluating the quality of the multinomial sampling watermark in more depth, rather than comparing the two.

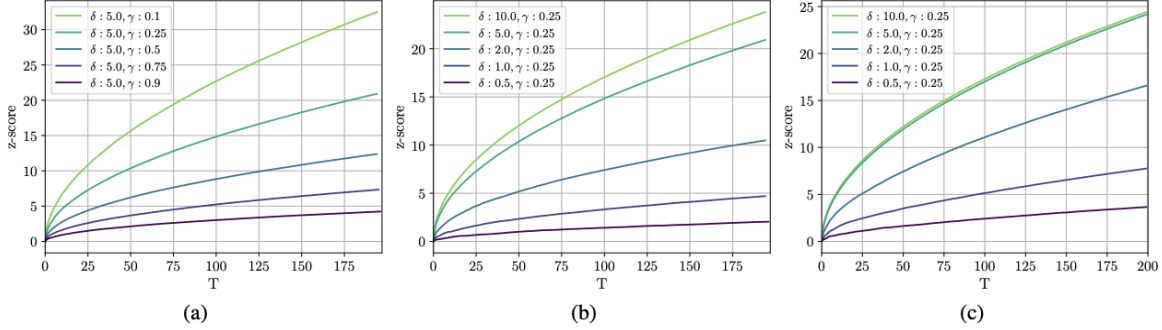


Figure 3. The average z -score as a function of T the token length of the generated text. (a) The dependence of the z -score on the green list size parameter γ , under multinomial sampling. (b) The effect of δ on z -score, under multinomial sampling. (c) The impact of the green list size parameter γ on the z -score, but with greedy decoding using 8-way beam search.

We also attempted to reproduce the below table, which tests the false positive/false negative rates for identifying watermarked text. This is important as if we want to use this in applications where we mark text as AI generated (e.g. flagging plagiarism), we must be very confident in the results of the detection algorithm.

sampling	δ	γ	count	$z=4.0$				$z=5.0$			
				FPR	TNR	TPR	FNR	FPR	TNR	TPR	FNR
m-nom.	1.0	0.50	506	0.0	1.0	0.767	0.233	0.0	1.0	0.504	0.496
m-nom.	1.0	0.25	506	0.0	1.0	0.729	0.271	0.0	1.0	0.482	0.518
m-nom.	2.0	0.50	507	0.0	1.0	0.984	0.016	0.0	1.0	0.978	0.022
m-nom.	2.0	0.25	505	0.0	1.0	0.994	0.006	0.0	1.0	0.988	0.012
m-nom.	5.0	0.50	504	0.0	1.0	0.996	0.004	0.0	1.0	0.992	0.008
m-nom.	5.0	0.25	503	0.0	1.0	1.000	0.000	0.0	1.0	0.998	0.002
8-beams	1.0	0.50	495	0.0	1.0	0.873	0.127	0.0	1.0	0.812	0.188
8-beams	1.0	0.25	496	0.0	1.0	0.819	0.181	0.0	1.0	0.770	0.230
8-beams	2.0	0.50	496	0.0	1.0	0.992	0.008	0.0	1.0	0.984	0.016
8-beams	2.0	0.25	496	0.0	1.0	0.994	0.006	0.0	1.0	0.990	0.010
8-beams	5.0	0.50	496	0.0	1.0	1.000	0.000	0.0	1.0	1.000	0.000
8-beams	5.0	0.25	496	0.0	1.0	1.000	0.000	0.0	1.0	1.000	0.000

Table 2. Empirical error rates for watermark detection using multinomial sampling and beam search. Each row is averaged over ~ 500 generated sequences of length $T = 200 \pm 5$. A maximum of one type-I (false positive) error was observed for any given run. All soft watermarks at $\delta = 2.0$ incur at most 1.6% (8/500) type-II error at $z = 4$. No type-II errors occurred for the hardest watermarks with $\delta = 10.0$ and $\gamma = 0.25$.

3. Methodology

We implemented the soft watermark from the paper as a wrapper on top of next-token generation, without changing model weights. At each step, we use the previous token and fixed hash key to pseudorandomly split the vocabulary into a green list and a red list, where the green list contains a fraction γ of the vocabulary. We then add a bias δ to the logits of the green list tokens before sampling. For detection, we reconstruct the same green list at each position, count how many generated tokens fall in the green list, and compute the z -score from the paper to decide whether the text is watermarked.

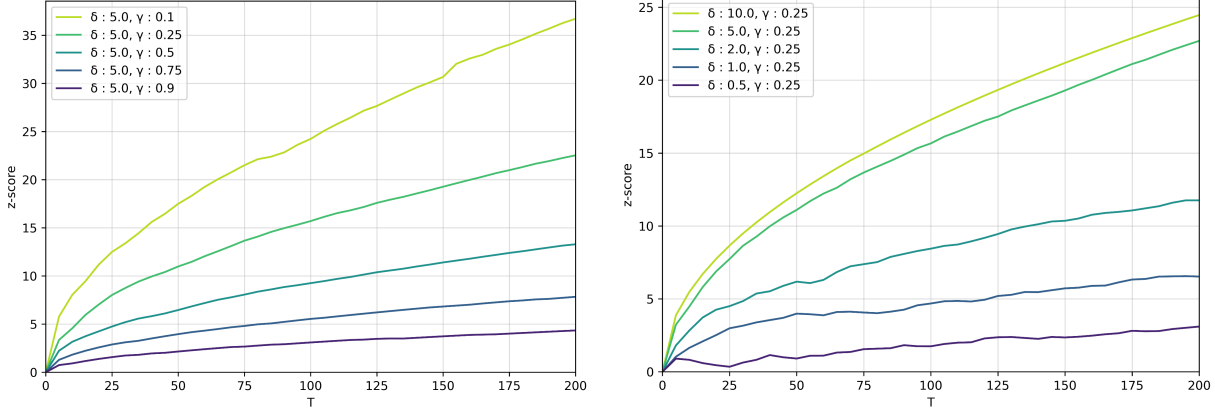
To reproduce the results from the original paper that we noted above, we ran our watermarking algorithm on top of OPT-1.3B, the same model used to generate these figures in the paper. For the prompts given to the model for token generation, we randomly selected them from the c4 dataset, the same as in the original paper.

In addition to the results we reproduced, we also created a “watermark detection game.” This is an original contribution of our own to empirically evaluate the quality of watermarked text. In this game, we used llama-3.2-3B for text generation. We allow the user to choose hyperparameters γ , δ , and then generate two completions for a random prompt from the pool: one using the model unchanged, and one running the model with the soft green list watermark. The user then has to attempt to guess which text

was watermarked. If the watermark causes a noticeable decrease in text quality (e.g. picks tokens that don't make as much sense because it is prioritizing a random subset of the vocabulary), then users should be able to identify it with higher probability. If users are not able to consistently pick out the watermark, this would show that most of the time the watermarked text is not noticeably lower quality.

4. Results & Analysis

Our graphs of the z-score versus token length look very similar to the results from the paper. Ours do show some amount of noise, but this is simply because we only ran 10 trials per set of parameters and only computed the z-score for token values that were multiples of 5, and decided to show our exact results rather than fit them along a curve (we felt this would be more authentic to our implementation).



Our table of the false positive and false negative rates shows similar results to that of the paper. We have a much smaller sample size, since this requires a lot of text generation and was more time consuming than we expected, likely due to sample size variance. However, the general trends remain the same: we see very few false positives, and our false negative rate is relatively low and decreases as δ increases.

sampling	δ	γ	count	$z = 4.0$				$z = 5.0$			
				FPR	TNR	TPR	FNR	FPR	TNR	TPR	FNR
m-nom.	1.0	0.50	20	0.000	1.000	0.789	0.211	0.000	1.000	0.706	0.294
m-nom.	1.0	0.25	20	0.024	0.976	0.944	0.056	0.000	1.000	0.941	0.059
m-nom.	2.0	0.50	20	0.000	1.000	1.000	0.000	0.000	1.000	1.000	0.000
m-nom.	2.0	0.25	20	0.024	0.976	1.000	0.000	0.000	1.000	1.000	0.000
m-nom.	5.0	0.50	20	0.000	1.000	1.000	0.000	0.000	1.000	1.000	0.000
m-nom.	5.0	0.25	20	0.000	1.000	1.000	0.000	0.000	1.000	1.000	0.000

Table 1: Empirical error rates for watermark detection using multinomial sampling. Each row is averaged over ~ 20 generated sequences. Created in LaTeX.

Analyzing the data collected by our watermarking game, we see that with the default watermark parameters ($\gamma = 0.5, \delta = 2$) users correctly identified the watermarked text in 188 out of 300 rounds (62.6%). This is above chance, but still fairly close to 50%, suggesting that with weaker watermark settings the quality difference between watermarked and non-watermarked text was often difficult to see. With the strongest settings we tested ($\gamma = 0.9, \delta = 5$), users correctly identified the watermarked text in 89 out of 100 rounds. This shows that stronger watermarking makes the watermark much easier for humans to notice, especially with the relatively weak model we used. Overall, these results suggest that our implementation was not perfect at hiding the watermark from human readers, but with proper parameter choices and a strong enough model, watermarking becomes difficult and may become impossible for a human reader to detect.

5. Reflections

One of the main takeaways from this project is that there is a clear tradeoff between detectability by an algorithm and detectability by a human. Stronger watermark parameters make the watermark easier to detect statistically, but they also make it easier for people to notice that something feels unnatural about the text. Some strategies that improved our ability to detect the watermark include large instances of white space, or the model restarting the sentence when the original would continue it. It would be useful to conduct future research testing how it is easier / harder to detect the algorithm from larger models, or prompts across different levels of perplexity. Another angle could be to implement the [watermark](#) that professor John Thickstun worked on, which we learned about while researching for this project.

6. References

- Grattafiori, A., et al. (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*. <https://arxiv.org/abs/2407.21783>
- Kirchenbauer, J., Geiping, J., Wen, Y., Katz, J., Miers, I., & Goldstein, T. (2023). A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning* (pp. 17061–17084). PMLR. <https://arxiv.org/abs/2301.10226>
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140), 1–67. <https://jmlr.org/papers/v21/20-074.html>
- Zhang, S., Roller, S., Goyal, N., Arber, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., Mihaylov, T., Ott, M., Shleifer, S., Shuster, K., Simig, D., Koura, P. S., Sridhar, A., Wang, T., & Zettlemoyer, L. (2022). OPT: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*. <https://arxiv.org/abs/2205.01068>